

Algorithmic Game Theory: Interview Guide

This guide connects strategic behavior with algorithms, complexity, and system design. Unless stated otherwise, players are rational, utilities are numerical, and a unilateral deviation changes only one player's action.

Computational Nash Equilibrium

1. Overview

A **computational Nash equilibrium** is a Nash equilibrium viewed as an algorithmic output: given a compact description of a game, compute a strategy profile in which no player can gain by deviating alone. In pure strategies this is a discrete search problem; in mixed strategies the output is a probability distribution for each player.

It matters because existence does not imply efficient findability. Nash's theorem guarantees a mixed equilibrium in every finite game, yet finding one in a general game is **PPAD-complete**. The topic appears in automated markets, multi-agent systems, security games, traffic assignment, and adversarial learning. Interviewers use it to test whether candidates can separate mathematical existence, representation size, verification, approximation, and computational complexity.

2. Core Idea

Think of an equilibrium as a fixed point of best responses. An algorithm receives payoffs, proposes strategies, computes every player's best attainable payoff against the others, and checks whether any improvement exists.

For a two-player matrix game, let row use distribution x and column use y . The profile is an **epsilon**-Nash equilibrium when neither can improve expected payoff by more than **epsilon**:

$$\max_i e_i^T A y - x^T A y \leq \epsilon \text{ and } \max_j x^T B e_j - x^T B y \leq \epsilon.$$

Example: in matching pennies there is no pure equilibrium. If each player chooses either action with probability $1/2$, every pure action gives the same expected payoff, so neither player benefits from changing probabilities.

Step by step: encode the game; choose pure or mixed output; find candidate supports or solve an appropriate optimization/fixed-point formulation; calculate deviation gains; return the profile only if all gains meet the required tolerance.

3. Important Subtopics

Representation and input size

Normal-form payoff tables grow exponentially with the number of players. Graphical, congestion, and polymatrix games exploit structure. This matters because an algorithm polynomial in the table size may still be

exponential in the number of players. Interviewers often ask what the input representation is before accepting a complexity claim.

Pure-equilibrium search

A pure profile can be verified by checking every unilateral deviation. Finding one is not always possible because a finite game may have none. For two players with m and n actions, scanning all cells and marking mutual best responses takes $O(mn)$ time.

Mixed equilibria and support enumeration

Actions in a player's support must yield equal maximum expected payoff. Support enumeration guesses supports, solves indifference equations, then verifies off-support inequalities. It is useful for small games but can examine exponentially many support pairs.

PPAD and total search

PPAD captures problems whose solutions are guaranteed through parity/fixed-point arguments. General Nash computation is PPAD-complete, which is evidence against a polynomial-time exact algorithm, though it is not the same as NP-completeness. This distinction is a frequent advanced interview test.

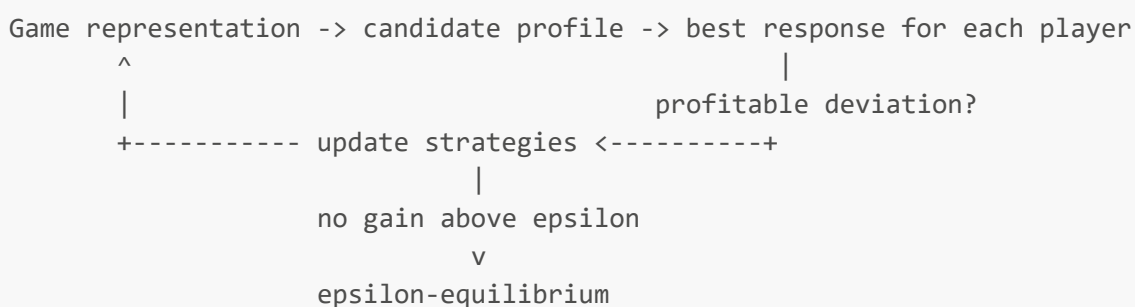
Approximate equilibrium

An ϵ -equilibrium permits deviation gain at most ϵ . Approximation is practical when inputs are noisy or exact probabilities require high precision, but the guarantee must specify additive versus multiplicative error and the utility scale.

4. Real-World Example

An ad platform repeatedly chooses reserve prices while advertisers choose bidding policies. A simulation service models these agents and seeks an approximate equilibrium. It computes expected utility under candidate mixed policies, runs best-response oracles, and reports the largest deviation gain. The platform can then tell whether the simulated market is strategically stable rather than merely profitable in one historical trace.

5. Diagrams / Mental Models



Question	Meaning
Does a solution exist?	Mathematical existence
Can it be checked quickly?	Verification complexity
Can it be found quickly?	Search complexity
Is near-stability enough?	Approximation guarantee

6. Common Interview Questions

1. **What is being computed?** A pure or mixed profile with no profitable unilateral deviation. **Expected:** state the game representation and equilibrium type. **Mistake:** saying only "the optimal strategy."
2. **Does every finite game have a Nash equilibrium?** Every finite game has a mixed equilibrium, not necessarily a pure one. **Expected:** cite the pure/mixed distinction. **Mistake:** claiming a pure equilibrium always exists.
3. **How do you verify a pure equilibrium?** Hold opponents fixed and test every alternative action for each player. **Expected:** $O(\text{number of unilateral deviations})$. **Mistake:** comparing social welfare.
4. **How do you find pure equilibria in a matrix?** Mark each player's best responses; intersections are equilibria. **Expected:** handle ties. **Mistake:** selecting the largest payoff sum.
5. **Why is mixed equilibrium harder?** Probabilities must satisfy nonlinear-looking mutual optimality and unknown supports. **Expected:** indifference plus off-support constraints. **Mistake:** averaging pure equilibria.
6. **What is PPAD-completeness?** Completeness for a class of guaranteed-solution search problems based on parity arguments. **Expected:** distinguish it from NP-completeness. **Mistake:** saying it proves exponential time.
7. **What is an ϵ -Nash equilibrium?** No unilateral deviation improves utility by more than ϵ . **Expected:** define the utility scale. **Mistake:** saying strategies are within ϵ probability distance.
8. **How does support enumeration work?** Guess supported actions, solve equal-payoff equations, then verify inequalities. **Expected:** mention exponential worst case. **Mistake:** checking only supported actions.
9. **Can equilibrium verification be easier than finding?** Yes; a supplied pure profile is easy to verify even when discovery is hard. **Expected:** search-versus-verification distinction. **Mistake:** assuming identical complexity.
10. **Why do best-response dynamics not solve every game?** They may cycle or fail to converge. **Expected:** mention special convergence classes such as potential games. **Mistake:** assuming utility improvement by one player gives global progress.

7. Deep-Dive Questions

1. **Why can exact mixed equilibria require care with arithmetic?** Equilibrium probabilities may be rational with large encodings; algorithms must define exact symbolic output or numerical tolerance.
2. **What is a best-response oracle?** Given opponents' strategies, it returns an action maximizing one player's expected utility; compact games often expose this without expanding the full payoff table.
3. **How do zero-sum games differ computationally?** Minimax equilibrium can be expressed as linear programs and solved in polynomial time, unlike general bimatrix Nash.

4. **What does degeneracy do?** Tied best responses can create extra equilibria, singular support equations, and duplicate outputs; verification remains essential.
5. **Could learning dynamics find equilibrium?** No-regret learning guarantees convergence of empirical play to coarse correlated equilibrium in broad settings, not necessarily Nash equilibrium.

8. Comparison Tables

Concept	Pure NE	Mixed NE	epsilon-NE
Output	One action/player	Distribution/player	Approximate profile
Existence in finite games	Not guaranteed	Guaranteed	Guaranteed via exact NE
Typical method	Best-response scan	Supports/fixed point	Iterative approximation
Verification	Direct deviations	Expected-payoff inequalities	Maximum regret $\leq$ epsilon
General-sum bimatrix		Zero-sum matrix	
Nash computation is PPAD-complete		Solvable by linear programming	
Players' objectives need not oppose exactly		One payoff is the negative of the other	

9. Common Mistakes

- Confusing equilibrium with maximum total payoff.
- Ignoring input representation when stating complexity.
- Claiming PPAD-complete means NP-complete or "no algorithm exists."
- Forgetting off-support actions during mixed-strategy verification.
- Reporting floating-point probabilities without a regret check.

10. Edge Cases / Special Cases

- Ties make best responses set-valued and may create continua of equilibria.
- Dominated actions normally receive zero probability, but weak domination requires care.
- Scaling utilities also scales an additive **epsilon**; normalize before comparing results.
- A timeout from an equilibrium solver is not evidence that no equilibrium exists.
- Succinct games can hide exponentially many pure profiles.

11. How to Explain in Interview

"Computational Nash equilibrium asks for an algorithm that turns a game description into mutually stable strategies. Pure candidates are easy to verify by checking unilateral deviations, while general mixed-equilibrium computation is PPAD-complete. In practice I specify the representation and tolerance, compute candidates using game structure, and validate them by maximum deviation gain."

12. Quick Revision Notes

- NE = zero unilateral regret; **epsilon**-NE = regret at most **epsilon**.
- Finite games guarantee mixed, not pure, equilibrium.
- General Nash: PPAD-complete; two-player zero-sum: linear programming.
- Supported actions are best responses and therefore have equal expected payoff.
- Trap: existence does not imply efficient computation.

13. Practice Tasks

1. Write a C++ function that marks mutual best responses in an $m \times n$ bimatrix.
2. Solve matching pennies using indifference equations and verify every deviation.
3. Implement a function returning maximum unilateral regret for a mixed profile.
4. Compare best-response dynamics on matching pennies and a potential game.
5. Explain why a polynomial algorithm in normal-form input may be exponential in player count.

14. Final Cheat Sheet

Item	Recall
Core definition	Compute a profile with no profitable unilateral deviation
Why it matters	Strategic stability must be algorithmically obtainable
Most asked	Existence, pure scan, support enumeration, epsilon -NE, PPAD
Key comparison	General-sum PPAD-complete vs zero-sum LP-solvable
One-line answer	"Find mutually optimal responses, then certify them by deviation gain."

Price of Anarchy

1. Overview

The **Price of Anarchy (PoA)** measures how inefficient selfish behavior can be in the worst equilibrium compared with a centrally optimal outcome. For a cost-minimization game,

$$\text{PoA} = (\text{largest social cost among equilibria}) / (\text{minimum possible social cost}).$$

For welfare maximization the ratio is conventionally reversed: **optimal welfare** / **smallest equilibrium welfare**, keeping values at least 1. PoA matters in network routing, load balancing, cloud resource sharing, protocol design, and auctions. Interviewers ask it to test modeling: define players, strategies, individual cost, social objective, equilibrium set, and ratio direction correctly.

2. Core Idea

PoA is a worst-case "cost of decentralization." Imagine drivers selfishly minimizing travel time. A traffic controller could route everyone efficiently, but independent drivers may settle at a stable pattern with higher total latency.

Pigou example: one unit of traffic chooses between an edge with latency $l(x)=x$ and an edge with constant latency 1. Selfish users can all take the variable edge, each seeing latency 1, so equilibrium total cost is 1. The

optimum sends half on each edge: cost $0.5^2 + 0.5 = 0.75$. Thus PoA is $1 / 0.75 = 4/3$.

The steps are: define social cost; characterize every equilibrium; find the worst equilibrium cost; compute the global optimum without equilibrium constraints; take the correctly oriented ratio.

3. Important Subtopics

Social cost and welfare

Social cost is often the sum of individual costs, though makespan, maximum latency, or weighted welfare may be used. The metric determines the result, so interviews expect it to be stated before calculation.

Worst-case equilibrium

PoA uses the worst equilibrium, not the equilibrium reached by a particular algorithm. This gives a robust guarantee when equilibrium selection is unknown.

Smoothness framework

A game is (λ, μ) -smooth when deviations toward an optimum satisfy a global inequality bounding total cost. Under standard conditions this yields $\text{PoA} \leq \lambda / (1 - \mu)$. Smoothness is valuable because its bounds extend to coarse correlated equilibria produced by no-regret learning.

Atomic versus nonatomic games

Atomic players control indivisible jobs or packets; nonatomic players represent infinitesimal traffic. A single atomic deviation can change congestion significantly, so bounds and equilibrium definitions differ.

Unbounded PoA

If selfish incentives can produce arbitrarily worse outcomes as instances scale, PoA is unbounded. A finite-looking example does not establish a universal bound; one must quantify over the instance family.

4. Real-World Example

In a cloud cluster, teams select servers to minimize their own job latency. The platform's objective is total completion time. PoA compares the worst stable self-selected assignment with the scheduler's optimal assignment. A high PoA justifies admission controls, prices, or a coordinating scheduler; a low bound says decentralization is safe even without predicting the selected equilibrium.

5. Diagrams / Mental Models

```

all feasible outcomes
|---- social optimum (best) ---- equilibria ---- worst equilibrium
      cost = OPT                      cost = WNE

PoA = WNE / OPT
  
```

Ratio value	Interpretation for cost minimization
1	Every equilibrium considered is optimal
1.2	Worst equilibrium costs at most 20% extra
Unbounded	No instance-independent efficiency guarantee

6. Common Interview Questions

1. **Define PoA.** Worst equilibrium objective divided by optimum for costs. **Expected:** ratio direction and worst equilibrium. **Mistake:** using average equilibrium.
2. **Can PoA be below 1?** Not under the standard normalized convention. **Expected:** optimum bounds feasible equilibria. **Mistake:** mixing cost and welfare formulas.
3. **What does PoA = 1 mean?** The worst equilibrium is socially optimal, so all equilibria are optimal for that metric. **Expected:** metric qualification. **Mistake:** claiming the equilibrium is unique.
4. **Why worst-case?** It gives a guarantee independent of equilibrium selection. **Expected:** robustness. **Mistake:** treating it as predicted observed performance.
5. **Is PoA about existence?** No; it compares efficiency once relevant equilibria exist. **Expected:** separate stability and quality. **Mistake:** saying high PoA means no equilibrium.
6. **How does PoA differ from approximation ratio?** Approximation ratio evaluates an algorithm's output; PoA evaluates selfish equilibrium outcomes. **Expected:** identify the source of suboptimality. **Mistake:** equating players with an algorithm.
7. **Can taxes improve PoA?** Yes, tolls can align private cost with marginal social cost. **Expected:** incentive alignment. **Mistake:** assuming any positive toll helps.
8. **Why does the social objective matter?** Sum latency and maximum latency can rank outcomes differently. **Expected:** specify objective. **Mistake:** using player utility sum automatically.
9. **What is robust PoA?** A smoothness-derived bound applying beyond pure NE to broader equilibrium notions. **Expected:** mention coarse correlated equilibrium. **Mistake:** calling it empirical robustness.
10. **When is PoA unbounded?** When a family of instances has equilibrium/optimum ratios growing without limit. **Expected:** family-level argument. **Mistake:** division by zero without discussing normalization.

7. Deep-Dive Questions

1. **Why can adding capacity worsen equilibrium?** Braess's paradox: a new route changes incentives so all users choose a collectively harmful path.
2. **What assumptions let smoothness bounds transfer?** The deviation inequality must hold for all action profiles, enabling expectation under coarse correlated distributions.
3. **Does a good PoA imply fairness?** No; aggregate efficiency can coexist with severe individual inequality.
4. **How does incomplete information change PoA?** Bayesian PoA compares expected welfare/cost at Bayes-Nash equilibria with an expected optimum.
5. **Can a system designer optimize PoA?** Yes, by choosing tolls, priorities, allocation rules, or information policies while respecting incentives and operational constraints.

8. Comparison Tables

Measure	Equilibrium chosen	Question answered
Price of Anarchy	Worst	How bad can selfish stability be?
Price of Stability	Best	Is at least one good stable outcome available?
Approximation ratio	Algorithm output	How good is the algorithm?
Competitive ratio	Online output	How good without future knowledge?

9. Common Mistakes

- Reversing the ratio for welfare problems.
- Comparing against the best equilibrium instead of the unconstrained optimum.
- Failing to define aggregate cost.
- Calculating one instance and claiming a bound for an entire game class.
- Assuming a low PoA means equitable outcomes.

10. Edge Cases / Special Cases

- If $\text{OPT} = 0$, a cost ratio may be undefined; use additive loss or restrict instances.
- Mixed, correlated, and coarse correlated equilibria may have different worst cases.
- Nonunique optima do not affect the optimal objective value.
- Weighted players can invalidate unweighted bounds.
- A bound may depend on the latency-function class, such as affine versus polynomial.

11. How to Explain in Interview

"Price of Anarchy is the worst-case efficiency loss caused by selfish equilibrium behavior. For costs, I divide the cost of the worst equilibrium by the globally minimum cost. It separates stability from system quality and helps decide whether a decentralized system needs pricing or coordination."

12. Quick Revision Notes

- Cost PoA = worst equilibrium cost / optimum cost.
- Welfare PoA = optimum welfare / worst equilibrium welfare.
- PoA is at least 1 under standard conventions.
- Smoothness gives reusable bounds and often covers learning outcomes.
- Trap: worst equilibrium, not best or observed equilibrium.

13. Practice Tasks

1. Compute the $4/3$ Pigou bound from the two-link example.
2. Construct a family with unbounded PoA.
3. Compare sum-latency and makespan PoA for the same load-balancing instance.
4. Simulate selfish best responses and calculate the resulting ratio.
5. Design marginal-cost tolls for a simple parallel-link network.

14. Final Cheat Sheet

Item	Recall
Core definition	Worst equilibrium versus social optimum
Why it matters	Quantifies the cost of decentralization
Most asked	Formula, Pigou example, PoS difference, tolls, smoothness
Key comparison	PoA uses worst NE; PoS uses best NE
One-line answer	"PoA is how inefficient the worst stable selfish outcome can be."

Price of Stability

1. Overview

The **Price of Stability (PoS)** compares the best equilibrium with the global optimum. For cost minimization,

$$\text{PoS} = (\text{minimum social cost among equilibria}) / \text{OPT}.$$

It asks whether selfish behavior is compatible with an efficient stable outcome if the system can guide users toward the right equilibrium. It is used in network formation, routing, load balancing, protocol defaults, and coordination mechanisms. Interviewers ask it alongside PoA to test quantifiers: PoS uses the best equilibrium; PoA uses the worst.

2. Core Idea

Suppose a system has several stable configurations. One is excellent and another is terrible. PoA reports the terrible possibility; PoS reports the opportunity available under good initialization, communication, or equilibrium-selection rules.

Example: two stable server conventions have costs 10 and 30, while the unconstrained optimum costs 8. Then $\text{PoS} = 10/8 = 1.25$ and $\text{PoA} = 30/8 = 3.75$. The gap indicates that selection, not equilibrium itself, is the main problem.

Compute it by finding the social optimum, enumerating or characterizing equilibria, selecting the lowest-cost equilibrium, and dividing by the optimum.

3. Important Subtopics

Best equilibrium

"Best" is defined by the system's social objective, not by one player's payoff. It matters because players may disagree about which equilibrium is desirable.

Equilibrium selection

Defaults, recommendations, communication, and initial conditions may steer a system toward a good equilibrium without changing payoff rules. PoS measures the upside of such selection.

Potential functions

In potential games, a global minimum of an exact potential is a pure equilibrium. Comparing that potential-minimizing equilibrium's social cost with optimum often yields PoS bounds. Social cost and potential are related but generally not identical.

Network design games

Players choose paths and share edge costs. An optimal network may not be stable, but a nearby stable network can exist; PoS measures this gap and can be far smaller than PoA.

4. Real-World Example

A distributed service lets clients choose one of several compatible protocol versions. Multiple adoption equilibria exist. The operator cannot force clients but can publish a default and migration guide. PoS tells the operator how close the best stable convention is to the globally cheapest configuration; the PoA–PoS gap estimates how valuable good coordination is.

5. Diagrams / Mental Models

```

OPT <= best equilibrium <= ... <= worst equilibrium
    |                               |
    +-- Price of Stability          +-- Price of Anarchy
  
```

For cost games: $1 \leq \text{PoS} \leq \text{PoA}$ whenever ratios are defined.

6. Common Interview Questions

1. **Define PoS.** Best equilibrium cost divided by optimum cost. **Expected:** best and unconstrained optimum. **Mistake:** using worst equilibrium.
2. **Why is PoS useful?** It measures what coordination or equilibrium selection can achieve without eliminating selfish choice. **Expected:** distinguish selection from redesign. **Mistake:** assuming the best equilibrium occurs automatically.
3. **Can PoS exceed PoA?** No for the same game, objective, and equilibrium concept in cost settings. **Expected:** best cost $\leq$ worst cost. **Mistake:** comparing different metrics.
4. **Can PoS equal 1 while PoA is large?** Yes; one equilibrium is optimal while another is poor. **Expected:** multiple-equilibrium example. **Mistake:** assuming all equilibria become optimal.
5. **Can PoS be unbounded?** Yes, if every equilibrium becomes arbitrarily worse than optimum across instances. **Expected:** family argument. **Mistake:** confusing "best" with "good."
6. **Does PoS apply with a unique equilibrium?** Yes, and then PoS equals PoA. **Expected:** same equilibrium set extremum. **Mistake:** saying selection is required for definition.
7. **What is the optimum comparison point?** The best feasible outcome, whether stable or not. **Expected:** no incentive constraint on OPT. **Mistake:** best equilibrium as denominator.
8. **How can defaults affect PoS in practice?** They can select the good equilibrium but do not change the mathematical PoS. **Expected:** bound versus mechanism. **Mistake:** claiming PoS guarantees convergence.
9. **Why are potential games useful?** Potential minima guarantee pure equilibria and help bound the best one. **Expected:** potential need not equal social cost. **Mistake:** treating every local minimum as globally best.

10. **What if welfare is maximized?** Use optimal welfare divided by the largest equilibrium welfare under the at-least-one convention. **Expected:** state convention. **Mistake:** blindly applying cost ratio.

7. Deep-Dive Questions

1. **Can computing PoS be harder than finding one equilibrium?** Yes; it requires optimizing over the equilibrium set, potentially needing many equilibria or a combined optimization formulation.
2. **What does a large PoA/PoS gap suggest?** Equilibria include both good and bad outcomes, so selection policies may yield large gains.
3. **Can communication achieve the PoS outcome?** Not always; messages may lack credibility or players may have conflicting preferences.
4. **How do perturbations help selection?** Small payoff or noise perturbations can remove weak equilibria, but may select an equilibrium different from the socially best one.
5. **Why not optimize the potential directly?** The potential encodes unilateral incentives; its ordering may not match social cost, so a bound connecting them is required.

8. Comparison Tables

Feature	PoS	PoA
Equilibrium	Best	Worst
Assumption	Helpful selection	Adversarial/unknown selection
Design lesson	Coordinate toward good NE	Redesign incentives if bound is bad
Cost relationship	$\text{PoS} \leq \text{PoA}$	$\text{PoA} \geq \text{PoS}$

9. Common Mistakes

- Assuming the best equilibrium is the global optimum.
- Forgetting PoS is still a worst-case-over-instances measure when quoting a class bound.
- Mixing equilibrium concepts between numerator calculations.
- Claiming a good PoS guarantees an easy path to the good equilibrium.
- Comparing potential values directly with costs without proof.

10. Edge Cases / Special Cases

- Unique equilibrium implies $\text{PoS} = \text{PoA}$.
- No pure equilibrium makes pure-PoS undefined, though mixed-PoS may exist.
- Zero optimum creates ratio problems.
- The best mixed equilibrium may outperform every pure equilibrium under the chosen objective.
- A socially best equilibrium may be unacceptable to individual players relative to another equilibrium, complicating selection.

11. How to Explain in Interview

“Price of Stability compares the socially best Nash equilibrium with the unconstrained optimum. It tells us whether decentralization can work well if we can coordinate which equilibrium is reached. For cost games it lies between 1 and the Price of Anarchy.”

12. Quick Revision Notes

- Cost PoS = best equilibrium cost / OPT.
- $1 \leq \text{PoS} \leq \text{PoA}$ for the same cost model.
- $\text{PoS} = \text{PoA}$ when the equilibrium is unique.
- Low PoS plus high PoA means equilibrium selection is valuable.
- Trap: availability of a good equilibrium does not guarantee convergence to it.

13. Practice Tasks

1. Create a game with PoS 1 and PoA greater than 2.
2. Compute both measures for three server assignments.
3. Find the minimum-potential equilibrium in a small congestion game.
4. Discuss a default policy that selects a good protocol-adoption equilibrium.
5. Explain why finding one equilibrium does not compute PoS.

14. Final Cheat Sheet

Item	Recall
Core definition	Best equilibrium versus social optimum
Why it matters	Measures attainable efficiency under good selection
Most asked	Formula, relation to PoA, unique equilibrium, potential games
Key comparison	PoS optimistic; PoA pessimistic
One-line answer	"PoS measures how good the best stable selfish outcome can be."

Congestion Games

1. Overview

A **congestion game** has players choosing sets of shared resources, where each resource's cost depends on how many players use it. A player's cost is usually the sum of costs of resources in their chosen set. Examples include jobs choosing servers, packets choosing links, processes choosing locks, and customers choosing facilities.

Congestion games matter because they model decentralized load and possess strong structure: every finite congestion game has a pure Nash equilibrium through Rosenthal's potential function. Interviewers ask about modeling, potential functions, best-response convergence, and inefficiency.

2. Core Idea

The externality is congestion: your resource choice changes both your cost and others' costs. Consider two jobs choosing servers A and B. Each server costs 1 with one job and 3 with two. If both choose A, either job can move to B and reduce cost from 3 to 1. If they split, moving would create congestion and raise the mover's cost to 3, so each split is a pure equilibrium.

For unweighted players with resource latency $c_e(k)$, Rosenthal's potential is

$$\Phi(s) = \sum_e \sum_{k=1}^{x_e(s)} c_e(k).$$

When one player changes strategy, the change in Φ exactly equals that player's cost change. Therefore every improving move decreases a finite-valued potential; improvement dynamics must terminate at a pure equilibrium.

3. Important Subtopics

Singleton and network congestion games

In singleton games each strategy selects one resource. In network congestion games a strategy is a path, so its resources are edges. Network structure makes shortest-path algorithms useful for best responses.

Exact potential

An exact potential mirrors every unilateral cost change. Its finite descent proves existence and convergence of better-response dynamics. It is not generally equal to total social cost.

Atomic, weighted, and nonatomic models

Atomic players are indivisible; weighted players contribute different loads; nonatomic traffic consists of infinitesimal users. Rosenthal's exact result applies directly to finite unweighted congestion games, while variants need separate analysis.

Social cost and inefficiency

Common objectives are total latency and makespan. Stable assignments can be inefficient, motivating PoA/PoS analysis and congestion pricing.

Best-response dynamics

Repeatedly let a player take a cheaper strategy. Finite improvement terminates, but the number of steps may be large and asynchronous implementation must use current load information.

4. Real-World Example

Microservices choose database replicas. Replica latency increases with concurrent load, and a request's cost is the latency of its chosen replica. Clients periodically switch to a faster replica. The system is a singleton congestion game; a pure equilibrium is a load assignment where no client benefits from moving alone. This model explains both convergence and oscillations caused when many clients update simultaneously rather than sequentially.

5. Diagrams / Mental Models

```

players choose resource sets -> loads  $x_e$  -> latency  $c_e(x_e)$ 
      ^                               |
      |                               v
      +----- best responses <- individual costs
  
```

Improving unilateral move => potential decreases => finite termination

6. Common Interview Questions

1. **Define a congestion game.** Players choose resource subsets and resource cost depends on load.
Expected: additive player cost. **Mistake:** describing only traffic networks.
2. **Why does a pure NE exist?** Rosenthal's exact potential decreases with every improving move.
Expected: finiteness. **Mistake:** invoking Nash's mixed-existence theorem.
3. **What is the potential formula?** Sum each resource's first through current-load marginal costs.
Expected: nested sum. **Mistake:** using $x_e c_e(x_e)$, which is social cost.
4. **Is potential equal to social cost?** Generally no. **Expected:** explain incentive versus welfare roles.
Mistake: assuming potential minimization is system optimization.
5. **What is a player's best response?** A feasible resource set minimizing their cost given current other-player loads. **Expected:** include the player's added load. **Mistake:** reading current edge costs without accounting for joining.
6. **Do best responses always converge?** Sequential strict improvements do in finite congestion games.
Expected: qualify update rule. **Mistake:** claiming simultaneous updates cannot cycle.
7. **What is a singleton congestion game?** Every strategy contains exactly one resource. **Expected:** server-selection example. **Mistake:** one player only.
8. **Atomic versus nonatomic?** Atomic players have finite impact; nonatomic users are infinitesimal flows.
Expected: different equilibrium concepts. **Mistake:** treating a packet and continuum identically.
9. **Can equilibrium be inefficient?** Yes; individual latency ignores external cost imposed on others.
Expected: connect to PoA. **Mistake:** assuming potential guarantees optimality.
10. **How can pricing help?** Charge marginal congestion externality to align private and social costs.
Expected: marginal-cost idea. **Mistake:** arbitrary flat fees.

7. Deep-Dive Questions

1. **Why is the Rosenthal sum exact?** Removing or adding a player changes only the last included cost term on each affected resource, matching that player's experienced cost difference.
2. **What breaks with weights?** A weighted move changes load by more than one, so the unit-increment potential no longer necessarily matches individual cost change.
3. **Is finding a pure equilibrium always fast?** Existence and terminating dynamics do not guarantee polynomial convergence; local-search complexity can be high.
4. **How does a network best response reduce to shortest path?** Fix others' edge loads and give each edge the latency after this player joins; minimum-cost feasible path is shortest.
5. **What does simultaneous migration cause?** Many players may chase the same currently cheap resource, overshoot, and oscillate; randomized or damped updates help operationally.

8. Comparison Tables

Model	Player size	Strategy	Equilibrium condition
Singleton congestion	Atomic	One resource	No cheaper resource
Network congestion	Atomic	Path	No cheaper path

Model	Player size	Strategy	Equilibrium condition
Nonatomic routing	Infinitesimal	Flow paths	Used paths have minimum latency
Potential	Social cost		
Tracks unilateral incentives	Measures system objective		
Guarantees pure NE existence	Defines efficiency/PoA		
Local minimum is NE	Minimum is social optimum		

9. Common Mistakes

- Using social cost as Rosenthal's potential.
- Forgetting a moving player's effect on destination load.
- Assuming all congestion games are network routing games.
- Applying unweighted results to weighted players without proof.
- Claiming potential descent gives a polynomial-time algorithm.

10. Edge Cases / Special Cases

- Constant resource costs remove the congestion externality.
- Tied moves are not strict improvements and can cause cycling if permitted carelessly.
- Nonmonotone latency functions can model discounts but invalidate standard intuition.
- Parallel updates need not preserve potential descent.
- Capacity constraints may make some deviations infeasible.

11. How to Explain in Interview

"A congestion game models players choosing shared resources whose costs grow with load. Its key property is Rosenthal's exact potential: every unilateral cost improvement lowers one global scalar, so finite unweighted congestion games always have a pure Nash equilibrium. The equilibrium may still be socially inefficient."

12. Quick Revision Notes

- Strategy = resource set; player cost = sum of loaded resource costs.
- Rosenthal potential sums marginal resource costs up to current load.
- Sequential strict better responses terminate at a pure NE.
- Potential is not social cost.
- Trap: weighted, nonatomic, and simultaneous-update variants need care.

13. Practice Tasks

1. Compute the potential for three jobs assigned to two servers.
2. Simulate sequential best responses and prove termination from the potential values.
3. Implement equilibrium verification for singleton resource choices.
4. Turn a path-selection instance into a congestion game.
5. Find an equilibrium that is worse than the minimum-total-latency assignment.

14. Final Cheat Sheet

Item	Recall
Core definition	Shared-resource game with load-dependent costs
Why it matters	Models decentralized resource contention
Most asked	Potential formula, pure existence, best responses, PoA
Key comparison	Potential tracks incentives; social cost tracks welfare
One-line answer	"Congestion games have pure equilibria because improving moves decrease Rosenthal's potential."

Network Routing Games

1. Overview

A **network routing game** models selfish users selecting source-to-destination paths in a network. Edge latency depends on total traffic, and each user minimizes the latency of their own path. It is a structured congestion game used for road traffic, packet routing, content delivery, distributed networks, and cloud traffic engineering.

The central question is whether individually shortest routes create an efficient global flow. Interviewers ask routing games because they combine graphs, shortest paths, equilibrium, convex optimization, and the counterintuitive Braess paradox.

2. Core Idea

In a **nonatomic** model, each user is infinitesimal and cannot change latency alone. A Wardrop equilibrium satisfies: for every source-destination pair, all used paths have equal minimum latency; unused paths have no smaller latency.

Example: one unit of traffic chooses two parallel edges with latencies $l_1(x)=x$ and $l_2(x)=1$. If all traffic uses edge 1, both edges cost 1, so no infinitesimal user can improve: this is equilibrium. The optimum splits traffic equally, giving total latency $0.5^2 + 0.5 = 0.75$. Selfish routing costs 1, producing PoA $4/3$.

Step by step: assign flows; sum flows to get edge loads; evaluate latencies; sum edge latencies along paths; move flow from a higher-latency used path to a lower one until Wardrop conditions hold.

3. Important Subtopics

Atomic versus nonatomic routing

Atomic users control whole packets, jobs, or demands and can noticeably change edge loads. Nonatomic models describe a continuum of negligible users. Atomic equilibrium uses unilateral path deviations; nonatomic equilibrium uses equal-latency used paths.

Wardrop equilibrium

Wardrop's first principle is the equilibrium condition for selfish flow. For separable nondecreasing edge latencies, a flow is Wardrop equilibrium exactly when it minimizes the Beckmann potential $\sum_e \int_0^{x_e} l_e(z) dz$ subject to flow constraints.

System-optimal routing

The system optimum minimizes total latency $\sum_e x_e l_e(x_e)$. Its marginal edge cost is $l_e(x) + x l'_e(x)$, which includes the delay an extra user imposes on existing traffic.

Braess's paradox

Adding a zero-cost or fast link can worsen equilibrium latency because it creates an attractive route whose combined congestion harms everyone. More capacity expands strategies but can damage a noncooperative equilibrium.

Tolls and marginal-cost pricing

A Pigouvian toll $x l'_e(x)$ makes a user's perceived edge cost equal the marginal social cost. Under suitable assumptions, selfish routing under these tolls implements a system optimum.

4. Real-World Example

A backend gateway chooses among service paths through caches, compute pools, and databases. Each path's latency rises with traffic. If every request selects its currently fastest path, the resulting Wardrop-like flow can overload a shared downstream dependency. Traffic engineering uses centrally assigned weights or congestion prices to internalize that externality.

5. Diagrams / Mental Models

```

      /-- edge A: latency x --\
source ----<                      >---- destination
      \-- edge B: latency 1 --/

```

Equilibrium: used paths have minimum equal latency.
 Optimum: route flow to minimize $\sum_e x_e l_e(x_e)$.

6. Common Interview Questions

1. **What is a routing game?** Users choose paths and edge delays depend on load. **Expected:** player objective and shared edges. **Mistake:** treating latency as fixed.
2. **Define Wardrop equilibrium.** Every used path has minimum latency for its commodity. **Expected:** unused paths cannot be cheaper. **Mistake:** requiring all network paths to have equal latency.
3. **Atomic versus nonatomic?** Atomic users affect loads discretely; nonatomic users are infinitesimal. **Expected:** different deviation definitions. **Mistake:** assuming results transfer automatically.
4. **What is total latency?** $\sum_e x_e l_e(x_e)$, equivalently flow-weighted path latency. **Expected:** multiply latency by flow. **Mistake:** sum edge latency once.
5. **Why is equilibrium not optimal?** Users consider personal travel time, not congestion imposed on others. **Expected:** externality. **Mistake:** blaming inaccurate shortest paths.

6. **What is Braess's paradox?** Adding capacity can worsen selfish equilibrium. **Expected:** changed incentives. **Mistake:** claiming physical capacity itself slows traffic.
7. **How are tolls chosen?** Charge marginal external cost, commonly $x \cdot l'_e(x)$. **Expected:** assumptions and units. **Mistake:** using total latency as toll.
8. **How is Wardrop equilibrium computed?** Minimize the Beckmann potential for separable monotone latency. **Expected:** flow conservation constraints. **Mistake:** minimizing total latency instead.
9. **What is the affine-latency PoA?** $4/3$ for nonatomic selfish routing. **Expected:** specify model and latency class. **Mistake:** universalizing it to all routing games.
10. **Can used paths have different latencies?** Not at Wardrop equilibrium for the same commodity, except zero-measure/path feasibility details. **Expected:** commodity qualification. **Mistake:** comparing different origins and destinations.

7. Deep-Dive Questions

1. **Why are Beckmann and social objectives different?** One integrates private latency to encode equilibrium; the other multiplies latency by all flow to count aggregate delay.
2. **When is equilibrium unique?** Edge-load or latency vectors may be unique under strict monotonicity, while path-flow decompositions can remain nonunique.
3. **How do multiple commodities change the model?** Each commodity has its own endpoints and Wardrop condition, while all contribute to shared edge congestion.
4. **Can marginal tolls be negative?** With decreasing latency they can, though standard congestion models assume nondecreasing functions and nonnegative externalities.
5. **What if users value time differently?** Generalized cost combines latency and toll using heterogeneous value-of-time, complicating implementation and fairness.

8. Comparison Tables

Feature	Wardrop equilibrium	System optimum
Objective	Each used path individually shortest	Total latency minimum
Edge expression	$l_e(x)$	Marginal cost $l_e(x) + x \cdot l'_e(x)$
Computation	Beckmann potential	Social-cost optimization
Externality	Ignored	Included

9. Common Mistakes

- Applying Wardrop equal-latency logic across different commodities.
- Minimizing the Beckmann potential and calling it system optimum.
- Forgetting flow conservation.
- Quoting $4/3$ without saying nonatomic affine latencies.
- Assuming added edges can never hurt equilibrium.

10. Edge Cases / Special Cases

- Zero-flow paths may have equal latency without being used.
- Discontinuous or nonmonotone delays complicate existence and computation.
- Equilibrium path flows may be nonunique even when edge loads are unique.

- Unsplittable demands produce atomic games and discrete effects.
- Tolls can optimize efficiency while raising distributional concerns.

11. How to Explain in Interview

“A network routing game has selfish users choose paths whose edge delays depend on aggregate flow. In the nonatomic model, Wardrop equilibrium means every used path is shortest. It can exceed the system-optimal total latency because users ignore congestion externalities; marginal-cost tolls can align the two.”

12. Quick Revision Notes

- Wardrop: used paths are minimum-latency paths.
- Total latency: $\sum_e x_e l_e(x_e)$.
- Beckmann integral computes equilibrium, not social optimum.
- Pigou network gives affine nonatomic PoA $4/3$.
- Braess: more route choice can worsen equilibrium.

13. Practice Tasks

1. Solve Wardrop and optimal flows for two parallel links.
2. Implement latency and total-cost evaluation for a path-flow vector.
3. Draw the classic Braess network and calculate before/after equilibria.
4. Derive $l(x) + x l'(x)$ by differentiating $x l(x)$.
5. Model API requests with two shared downstream paths.

14. Final Cheat Sheet

Item	Recall
Core definition	Selfish path selection under congestion
Why it matters	Explains inefficient traffic and network load
Most asked	Wardrop, Pigou, Braess, PoA, marginal tolls
Key comparison	Private shortest path vs minimum total latency
One-line answer	“Used paths are individually shortest, but that flow need not minimize total delay.”

Matching Markets

1. Overview

A **matching market** assigns agents on one or more sides—workers to firms, students to schools, riders to drivers, or donors to recipients—while respecting preferences and constraints. Unlike pricing-only markets, who matches with whom is central. Desired properties include stability, feasibility, efficiency, fairness, and incentive compatibility.

Matching markets power residency placement, school choice, labor platforms, ad allocation, and kidney exchange. Interviewers ask them to test graph modeling, preference handling, deferred acceptance, invariants, and trade-offs between stability and optimality.

2. Core Idea

A matching is stable when no **blocking pair** prefers each other to their assigned outcomes. Such a pair could bypass the mechanism, so instability threatens participation.

Example: applicants A, B and jobs X, Y . $A: X > Y, B: X > Y; X: B > A, Y: A > B$. Deferred acceptance has A and B propose to X ; X tentatively holds B and rejects A ; A proposes to Y . Final matching $(B, X), (A, Y)$ has no blocking pair.

The algorithm delays final commitment: proposers apply to their best unrejected option; receivers retain their favorite offer so far; rejected proposers continue. Rejections are permanent, and the process ends after finitely many proposals.

3. Important Subtopics

One-to-one and many-to-one matching

Stable marriage pairs one agent per side. College admissions or job markets give receivers capacities. Deferred acceptance extends by letting each receiver retain its top q proposals.

Preferences, priorities, and acceptability

Preferences rank partners; schools often have priorities rather than strategic preferences. Unacceptable pairs must never be produced. Ties and incomplete lists require explicit stability definitions.

Stability and blocking pairs

A blocking pair consists of mutually preferred compatible agents. Stability prevents profitable bilateral rematching, but does not automatically maximize total rank or fairness.

Proposer optimality

In strict one-to-one preferences, proposer-side deferred acceptance returns the proposer-optimal stable matching and receiver-pessimal stable matching. Which side proposes is therefore a material design choice.

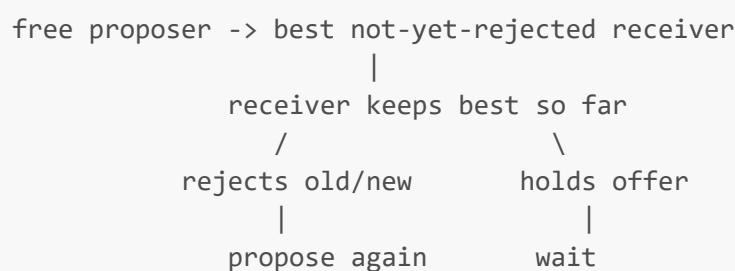
Strategy-proofness and efficiency

Proposer-side deferred acceptance is strategy-proof for individual proposers under the classical model, but not generally for receivers. Stable outcomes may fail Pareto efficiency when compared with unstable matchings.

4. Real-World Example

A distributed batch platform matches jobs to machines. Jobs rank machines by latency and data locality; machines rank jobs by priority, memory fit, and deadlines. A stable assignment prevents a job and machine from both preferring to abandon current partners. Capacities turn the model into many-to-one matching, while hard resource requirements define unacceptable pairs.

5. Diagrams / Mental Models



6. Common Interview Questions

1. **What is a matching market?** A system assigning agents under preferences and constraints. **Expected:** graph/matching view. **Mistake:** reducing it to monetary prices.
2. **What is stability?** No acceptable unmatched pair mutually prefers each other. **Expected:** blocking-pair definition. **Mistake:** equating stability with maximum cardinality.
3. **Why deferred acceptance?** It constructs a stable matching while delaying irreversible choices. **Expected:** proposals, tentative holds, rejections. **Mistake:** receivers accepting permanently on first offer.
4. **What is proposer optimality?** Every proposer weakly prefers the proposer-DA result to every other stable matching. **Expected:** restricted to stable outcomes. **Mistake:** global Pareto optimality.
5. **What is the running time?** $O(n^2)$ proposals in an $n \times n$ market with indexed rankings. **Expected:** each pair proposed at most once. **Mistake:** counting rounds only.
6. **Does stable matching always exist?** Yes in the classical bipartite strict-preference model, with unmatched options handled properly. **Expected:** qualify variants. **Mistake:** extending to roommates automatically.
7. **Can stable matchings differ?** Yes. **Expected:** proposing side affects which stable matching. **Mistake:** assuming DA output is unique.
8. **How do capacities change DA?** A receiver retains its best q proposals. **Expected:** reject beyond capacity. **Mistake:** cloning seats without considering coupled constraints.
9. **Are stable matchings socially optimal?** Not necessarily for total rank or cardinal welfare. **Expected:** separate stability and efficiency. **Mistake:** treating no blocking pair as maximum welfare.
10. **Who can manipulate?** Truth-telling is dominant for individual proposers in classical DA, not generally for receivers. **Expected:** model qualification. **Mistake:** claiming full strategy-proofness.

7. Deep-Dive Questions

1. **What is the rural hospitals phenomenon?** Under standard many-to-one assumptions, every stable matching fills the same number of positions at each hospital and matches the same set size, despite different pairings.
2. **How do ties affect stability?** Weak, strong, and super stability differ in what preference indifference permits; existence and complexity change.
3. **Why is stable roommates harder?** It is non-bipartite and a stable matching may not exist, requiring a different algorithm.
4. **What are substitutable preferences?** A receiver's desire for one contract should not increase merely because another is removed; this condition supports stable matching with contracts.

5. **How does kidney exchange differ?** Compatibility is primary and exchanges form cycles/chains; maximizing feasible transplants is a constrained graph optimization problem rather than classical two-sided preferences.

8. Comparison Tables

Property	Stable matching	Maximum-weight matching
Goal	No blocking pair	Maximize total edge weight
Input	Ordinal preferences	Cardinal weights
Participation stability	Strong	Not guaranteed
Typical algorithm	Deferred acceptance	Hungarian/min-cost flow
Proposer-side DA	Receiver-side DA	
Proposer-optimal stable result	Receiver-optimal stable result	
Strategy-proof for proposers	Strategy-proof for receivers in classical model	

9. Common Mistakes

- Calling any perfect matching stable.
- Checking only unmatched agents rather than unmatched pairs.
- Saying proposer optimal means globally optimal.
- Ignoring unacceptable partners and unmatched preferences.
- Assuming results survive ties, couples, or complementarities unchanged.

10. Edge Cases / Special Cases

- Unequal side sizes produce unmatched agents.
- Empty preference lists are valid when being unmatched is allowed.
- Ties require choosing a stability definition and tie-breaking policy.
- Couples create complementarities and may destroy existence.
- Capacity changes or regional quotas can break standard guarantees.

11. How to Explain in Interview

"Matching markets allocate partners under preferences. The key solution concept is stability: no unmatched pair should mutually prefer each other. Deferred acceptance achieves stability by letting proposers apply in order while receivers tentatively retain their best offers, and it returns the proposer-optimal stable matching in the classical model."

12. Quick Revision Notes

- Blocking pair = mutually preferred unmatched pair.
- DA: propose, tentatively hold, permanently reject.
- At most one proposal per pair: $O(n^2)$.
- Proposer-optimal among stable matchings.

- Trap: stable is not the same as maximum-weight or fair.

13. Practice Tasks

1. Implement proposer-side deferred acceptance with ranking arrays.
2. Write a blocking-pair verifier.
3. Run the same preferences with each side proposing.
4. Extend the implementation to receiver capacities.
5. Construct a stable matching with lower total rank than an unstable matching.

14. Final Cheat Sheet

Item	Recall
Core definition	Preference-based allocation without blocking pairs
Why it matters	Produces durable assignments
Most asked	DA steps, stability proof, complexity, proposer optimality
Key comparison	Stable matching vs maximum-weight matching
One-line answer	"Deferred acceptance prevents blocking pairs by making rejections final and acceptances tentative."

Stable Marriage

1. Overview

The **stable marriage problem** is the classical one-to-one bipartite matching problem: two equally sized groups rank members of the other group, and the goal is a perfect matching with no blocking pair. Names such as "men" and "women" are historical; algorithmically they are proposer and receiver sides.

It matters as the foundation of residency, admissions, and matching systems and as a clean example of algorithm design with invariants. Interviewers expect the Gale–Shapley algorithm, termination, stability proof, complexity, and proposer-optimality.

2. Core Idea

Gale–Shapley deferred acceptance works because receivers only trade up while proposers move monotonically down their preference lists. A rejected proposer can never later form a blocking pair with that receiver: the receiver already holds, and will thereafter hold, someone preferred.

Example: $P1: R1 \succ R2$, $P2: R1 \succ R2$; $R1: P2 \succ P1$, $R2: P1 \succ P2$. Both first propose to $R1$; $R1$ holds $P2$ and rejects $P1$; $P1$ proposes to $R2$. The matching $P2-R1$, $P1-R2$ is stable.

Proof outline: proposals terminate because no pair repeats. If final partners $p-r'$ and $p'-r$ were blocked by $p-r$, then p must have proposed to r before r' ; r rejected p for someone preferred and never traded down, contradiction.

3. Important Subtopics

Gale–Shapley data structures

Maintain a queue of free proposers, each proposer's next preference index, every receiver's current partner, and inverse receiver rankings for $O(1)$ comparison. This achieves $O(n^2)$ time and $O(n^2)$ preference storage.

Termination and stability proofs

Termination follows from at most n^2 distinct proposals. Stability follows from rejection monotonicity. Interviewers often want both proofs separately.

Optimality and lattice structure

Proposer-DA gives every proposer their best partner attainable in any stable matching and every receiver their worst stable partner. Stable matchings form a lattice under side preferences.

Uniqueness

The stable matching is unique only when proposer-optimal and receiver-optimal results coincide. Unique preferences do not imply a unique stable matching.

Variants

Incomplete lists allow unmatched agents; ties change stability definitions; unequal sides change perfectness; stable roommates removes bipartiteness and may lack a solution.

4. Real-World Example

A service marketplace pairs long-running client sessions with dedicated backend workers. Both sides rank compatible partners using latency, hardware features, and workload fit. Deferred acceptance creates a stable allocation that discourages a client-worker pair from bypassing the scheduler. In production, quotas and capacity make this a generalized matching market, but stable marriage remains the core interview model.

5. Diagrams / Mental Models

```

Free P -> propose to next R
          |
          v
R free? ----+---- yes -> hold P
          |
          no
          v
prefer P to current?
yes: swap; old P free
no: reject P
  
```

6. Common Interview Questions

1. **State the problem.** Find a one-to-one bipartite matching with no blocking pair. **Expected:** strict preferences and classical assumptions. **Mistake:** maximum-weight matching.
2. **Describe Gale–Shapley.** Free proposers apply in order; receivers hold their favorite proposal so far. **Expected:** tentative acceptance. **Mistake:** permanent first acceptance.
3. **Why does it terminate?** Each proposer-receiver pair occurs at most once. **Expected:** at most n^2 proposals. **Mistake:** “because preferences improve” without a finite bound.
4. **Why is it stable?** Any alleged blocking receiver must earlier have rejected that proposer for someone preferred. **Expected:** receiver never trades down. **Mistake:** proving only perfectness.
5. **Time complexity?** $O(n^2)$ with inverse rankings and next indices. **Expected:** distinguish input reading. **Mistake:** $O(n^3)$ from scanning receiver lists.
6. **Is the result unique?** Not necessarily. **Expected:** compare both proposing orientations. **Mistake:** strict preferences imply uniqueness.
7. **What does proposer-optimal mean?** Best stable partner for every proposer. **Expected:** among stable matchings. **Mistake:** first choice for everyone.
8. **Is it strategy-proof?** For individual proposers in the classical mechanism. **Expected:** not symmetric. **Mistake:** all agents always truthful.
9. **How do you verify stability?** Check every unmatched pair against both current partners using rank arrays. **Expected:** $O(n^2)$. **Mistake:** rerun DA as verification.
10. **What if list sizes differ?** Add unmatched handling or dummy least-preferred partners; stable matchings need not be perfect. **Expected:** acceptability. **Mistake:** force unacceptable matches.

7. Deep-Dive Questions

1. **How can uniqueness be tested?** Compute proposer-optimal and receiver-optimal stable matchings; if identical, the stable matching is unique.
2. **Why can receivers manipulate?** The mechanism's proposer strategy-proofness is asymmetric; a receiver may sometimes misreport to induce a preferred stable outcome.
3. **What are rotations?** Structured cycles of partner changes used to enumerate or optimize across all stable matchings.
4. **Does asynchronous proposal order change the result?** With fixed proposing side and strict preferences, no; proposer-optimal output is invariant.
5. **Why can stable roommates fail?** Without bipartition, preference cycles can prevent any matching from eliminating all blocking pairs.

8. Comparison Tables

Feature	Stable marriage	Stable roommates
Graph	Bipartite	General
Classical existence	Always	Not guaranteed
Basic algorithm	Gale–Shapley	Irving's algorithm
Concept	Meaning	
Perfect	Everyone matched	
Stable	No blocking pair	

Concept	Meaning
Proposer-optimal	Best stable partner for each proposer
Socially optimal	Maximizes chosen aggregate metric

9. Common Mistakes

- Making acceptance final.
- Comparing receiver preferences by scanning instead of inverse ranks.
- Claiming stable means everyone gets a top choice.
- Confusing proposer order with which side proposes.
- Forgetting to define how unmatched compares with unacceptable partners.

10. Edge Cases / Special Cases

- $n=0$ returns the empty stable matching.
- Duplicate or missing preference entries require validation.
- Ties invalidate the standard strict-comparison proof unless a stability convention is chosen.
- A stable matching may be perfect yet have poor aggregate rank.
- Proposal order does not change classical proposer-optimal output.

11. How to Explain in Interview

"Stable marriage asks for a bipartite one-to-one matching with no mutually preferable unmatched pair. Gale–Shapley lets free proposers apply in preference order while receivers keep their best offer so far. There are at most n^2 proposals, rejection monotonicity proves stability, and the result is proposer-optimal among stable matchings."

12. Quick Revision Notes

- Maintain free queue, next index, receiver match, inverse ranks.
- Termination: no repeated proposal.
- Stability: rejected proposers cannot later block.
- Proposing side gets its optimal stable result.
- Trap: strict preferences do not guarantee uniqueness.

13. Practice Tasks

1. Implement Gale–Shapley in C++ and assert stability.
2. Trace a 4×4 instance proposal by proposal.
3. Produce an example with two stable matchings.
4. Test uniqueness using both proposal directions.
5. Extend verification to incomplete lists.

14. Final Cheat Sheet

Item	Recall
Core definition	One-to-one bipartite matching without blocking pairs
Why it matters	Canonical stable-allocation algorithm
Most asked	Algorithm, proof, $O(n^2)$, optimality, uniqueness
Key comparison	Stability differs from perfectness and welfare
One-line answer	"Gale–Shapley is deferred acceptance with permanent rejections."

Mechanism Design

1. Overview

Mechanism design is reverse game theory: instead of predicting behavior under fixed rules, a designer chooses rules so self-interested behavior produces a desired outcome. A mechanism specifies messages, an allocation rule, and often payments. Core goals include incentive compatibility, individual rationality, efficiency, budget balance, and computational feasibility.

It is used in auctions, ad markets, cloud pricing, matching platforms, voting, bandwidth allocation, and API quotas. Interviewers ask it to see whether candidates can reason about incentives, not merely implement an allocation algorithm.

2. Core Idea

Agents possess private information, such as their value for an item. A naive rule may reward lying. A well-designed mechanism makes truthful reporting optimal or at least an equilibrium.

Single-item second-price auction: allocate to the highest bidder and charge the second-highest bid. If your value is v , bidding above v risks winning at a price above value; bidding below v risks losing when the price would have been below value. Reporting v is therefore weakly dominant.

Design steps: define types and utilities; choose desired social choice; define reports; specify allocation and payment rules; prove incentive compatibility and participation; check budget and computational constraints.

3. Important Subtopics

Incentive compatibility

Dominant-strategy IC makes truth best regardless of others' reports. Bayesian IC makes truth best in expectation over beliefs. DSIC is stronger and easier for users to reason about.

Individual rationality

An individually rational participant gets nonnegative utility compared with opting out. Ex-post IR holds after every realization; interim IR holds in expectation over unknowns.

VCG mechanisms

VCG chooses the welfare-maximizing allocation and charges each winner the externality imposed on others. Under quasilinear utilities it is DSIC and efficient, but may be computationally expensive and need not be budget-balanced.

Revelation principle

If an outcome can be implemented by some equilibrium mechanism, an equivalent direct truthful mechanism exists under the corresponding solution concept. It simplifies analysis, not implementation cost or communication automatically.

Impossibility and trade-offs

Desirable properties can conflict. Myerson–Satterthwaite shows bilateral trade cannot simultaneously achieve efficiency, Bayesian IC, individual rationality, and budget balance under standard private-value uncertainty.

4. Real-World Example

A cloud provider allocates limited GPU slots. Users privately value completion before deadlines. A mechanism collects bids, selects jobs maximizing reported welfare under capacity constraints, and charges critical payments. The allocation algorithm alone is insufficient: without suitable payments, users exaggerate values and distort scheduling.

5. Diagrams / Mental Models

```
private types -> reported messages -> allocation rule -> outcome
      |                               |
      +----- utility + payment <-----+
```

Designer asks: Is truthful reporting the best strategic response?

6. Common Interview Questions

1. **What is mechanism design?** Designing rules to induce desired outcomes under strategic private information. **Expected:** reverse-game-theory framing. **Mistake:** calling it ordinary optimization.
2. **What is DSIC?** Truth maximizes each agent's utility for every report profile of others. **Expected:** dominant strategy. **Mistake:** truth only at equilibrium.
3. **What is Bayesian IC?** Truth maximizes expected utility given beliefs about others' types. **Expected:** expectation and prior. **Mistake:** stronger than DSIC.
4. **What is individual rationality?** Participation utility is at least the outside option. **Expected:** state ex-post/interim if relevant. **Mistake:** equating it with social welfare.
5. **Why is second price truthful?** Bid controls whether you win; conditional price is the highest competing bid, not your own. **Expected:** overbid/underbid cases. **Mistake:** saying winners always pay their value.
6. **What does VCG charge?** The welfare loss your presence imposes on everyone else. **Expected:** externality formula. **Mistake:** simply second-highest bid in every setting.
7. **What is the revelation principle?** Any implementable outcome has an equivalent direct truthful mechanism. **Expected:** analytical reduction. **Mistake:** truthfulness is free in every practical system.

8. **Efficiency versus revenue?** Welfare maximization and seller revenue maximization are different objectives. **Expected:** VCG versus optimal auctions. **Mistake:** assuming highest welfare maximizes revenue.
9. **What is budget balance?** Payments collected cover payments made, exactly or weakly depending definition. **Expected:** distinguish no-deficit and exact balance. **Mistake:** confusing it with individual rationality.
10. **Why does computational complexity matter?** A truthful payment rule may require solving allocation optimization repeatedly. **Expected:** VCG externality computation. **Mistake:** proving incentives while ignoring infeasible allocation.

7. Deep-Dive Questions

1. **What is Myerson's lemma in a single-parameter domain?** DSIC requires a monotone allocation rule, with payments determined by a threshold/integral formula.
2. **Can an approximation algorithm remain truthful?** Not automatically; approximation can destroy allocation monotonicity. Maximal-in-range or monotone designs are common remedies.
3. **Why is VCG vulnerable in practice?** It can have low revenue, fail budget balance, require expensive optimization, and be sensitive to collusion or false identities.
4. **What is implementation?** A mechanism implements a social choice rule when its equilibria produce the intended outcomes under a specified equilibrium concept.
5. **Why distinguish values from bids?** Value is private type; bid is strategic message. Equating them assumes the truthfulness that must be proven.

8. Comparison Tables

Property	DSIC	Bayesian IC
Truth best against	Every report profile	Distribution of others' types
Prior required	No	Yes
Strength	Stronger	Weaker
First-price auction		Second-price auction
Winner pays own bid		Winner pays highest losing bid
Strategic bid shading		Truthful bidding is weakly dominant
Revenue depends on equilibrium		Simple DSIC argument

9. Common Mistakes

- Assuming reported bids equal true values before proving truthfulness.
- Confusing incentive compatibility with individual rationality.
- Calling VCG universally revenue-optimal.
- Ignoring tie-breaking, which can affect monotonicity.
- Treating computationally optimal allocation as automatically implementable.

10. Edge Cases / Special Cases

- Ties need a fixed report-independent rule.
- Negative values or externalities can break standard auction assumptions.
- Non-quasilinear utilities invalidate simple payment arguments.
- Collusion and false-name bids are not prevented by ordinary DSIC.
- Multi-parameter domains are substantially harder than single-parameter ones.

11. How to Explain in Interview

"Mechanism design chooses allocation and payment rules so strategic agents reveal useful private information and the resulting equilibrium meets a system objective. I check incentive compatibility, participation, efficiency, and budget balance separately; a second-price auction is the standard DSIC example."

12. Quick Revision Notes

- Mechanism = message space + allocation rule + payments.
- Utility commonly **value** - **payment** under quasilinearity.
- DSIC is pointwise; BIC is expectation over types.
- VCG: welfare-maximizing allocation, externality payment.
- Trap: allocation quality alone says nothing about truthful incentives.

13. Practice Tasks

1. Prove second-price truthfulness by overbid and underbid cases.
2. Compute VCG payments for two items and three bidders.
3. Test monotonicity of a single-parameter allocation algorithm.
4. Give separate examples violating IR and budget balance.
5. Design a critical-value payment for a simple job scheduler.

14. Final Cheat Sheet

Item	Recall
Core definition	Design rules so strategic behavior yields desired outcomes
Why it matters	Private information otherwise distorts allocation
Most asked	DSIC/BIC, IR, VCG, revelation principle, budget balance
Key comparison	Allocation optimization vs incentive-compatible implementation
One-line answer	"Mechanism design engineers incentives, not just outcomes."

Online Auctions

1. Overview

An **online auction** makes allocation and pricing decisions as bidders, items, impressions, or time slots arrive over time, often before future information is known. Decisions may be irrevocable and subject to inventory, budget, latency, and truthfulness constraints.

Online auctions drive ad exchanges, spot compute, ticketing, delivery platforms, and real-time marketplaces. Interviewers ask them to combine online algorithms, auctions, pacing, competitive analysis, randomized decisions, and production constraints.

2. Core Idea

The mechanism must decide now without seeing later opportunities. Selling an item to an early moderate bidder may lose a later high bidder; waiting may leave it unsold.

Example: one item, buyers arrive sequentially, values are private. A posted-price mechanism chooses price p before or during the process and sells to the first buyer with value at least p . A buyer has no bid-shading problem when the price is fixed independently of their report, but performance depends on how p is chosen from distributions, samples, or worst-case assumptions.

Step by step: define arrival and information model; define feasibility and irrevocability; choose objective such as welfare or revenue; allocate and price upon arrival; prove incentive properties; compare against an offline benchmark.

3. Important Subtopics

Arrival models

Adversarial order gives strong worst-case guarantees but may force randomization. Random-order and stochastic arrivals permit stronger results using sampling and learned distributions. The guarantee is meaningless unless the arrival model is stated.

Competitive analysis

The competitive ratio compares expected online performance with an offline optimum that knows the future. For maximization, conventions commonly state $\text{ALG} \geq \text{OPT}/c$. Unlike PoA, inefficiency comes from missing future information, not strategic equilibrium.

Posted prices and truthfulness

A take-it-or-leave-it price independent of the current buyer's report is transparently truthful. Dynamic posted prices adjust to time, remaining inventory, and prior observations while preserving the required independence.

Prophet inequalities and secretary models

Prophet inequalities compare an online stopping policy with an omniscient selector under known independent distributions. Secretary problems use random arrival order with unknown values and often a sample-then-select threshold.

Ad auctions, budgets, and pacing

Repeated impression auctions must respect advertiser budgets. Pacing multipliers scale bids to distribute spend, while reserve prices protect revenue. Per-auction truthfulness does not eliminate strategic behavior across time or budget reports.

4. Real-World Example

An ad exchange receives an impression and must respond in milliseconds. Eligible advertisers submit bids, but campaigns have remaining budgets and frequency caps. The system applies pacing, runs an auction, charges a price, logs the outcome, and updates budgets atomically. It cannot wait for future impressions, so it evaluates revenue and welfare against stochastic forecasts or offline replay benchmarks.

5. Diagrams / Mental Models

```

arrival t -> observe current bid/context -> decide allocate/reject + price
      |               |               |
      |               |               |
future hidden    remaining budget    usually irrevocable

offline benchmark sees: arrival 1 ... arrival T
  
```

6. Common Interview Questions

- What makes an auction online?** Decisions occur sequentially without complete future information.
Expected: arrival model and irrevocability. **Mistake:** merely being hosted on the internet.
- What is the benchmark?** Usually offline optimal welfare or revenue with full future knowledge.
Expected: same feasibility constraints. **Mistake:** comparing against unlimited inventory.
- What is a competitive ratio?** A worst-case bound between online and offline objectives under a stated convention. **Expected:** ratio direction. **Mistake:** confusing it with PoA.
- Why use posted prices?** They are simple, fast, and truthful when independent of the buyer's report.
Expected: accept/reject decision. **Mistake:** changing price after seeing the reported value.
- Secretary versus prophet inequality?** Secretary assumes random order, often no value distributions; prophet assumes known independent distributions. **Expected:** information distinction. **Mistake:** treating both as adversarial order.
- Why is randomization useful?** Deterministic algorithms can be exploited by adversarial sequences.
Expected: adversary model. **Mistake:** claiming randomness guarantees optimality.
- How do budgets change auctions?** Bids become coupled across time; spending now reduces future feasibility. **Expected:** pacing and state. **Mistake:** treating each auction independently.
- Is second price enough online?** It can make a single auction truthful, but does not solve inventory, timing, budget, or intertemporal incentives. **Expected:** separate layers. **Mistake:** assuming global optimality.
- What must be atomic in implementation?** Inventory/budget reservation and winner charging or rollback. **Expected:** avoid overselling/double spend. **Mistake:** update asynchronously without idempotency.
- How are online mechanisms evaluated?** Theory uses competitive/Bayesian guarantees; systems use replay, experiments, calibration, latency, and constraint violations. **Expected:** counterfactual caveats. **Mistake:** replaying bids as if behavior never changes.

7. Deep-Dive Questions

- What is the $1/e$ secretary strategy?** Observe roughly the first n/e arrivals without selecting, then take the next record; it succeeds with probability approaching $1/e$ in the classical model.

2. **How can learning and truthfulness conflict?** Using an agent's current bid to update their offered price lets reports manipulate future treatment; exploration must be designed carefully.
3. **What is bid shading in repeated first-price auctions?** Bidders reduce bids below value based on competition and budget opportunity cost; equilibrium behavior changes with pacing and feedback.
4. **Why is offline replay biased?** Historical bids and participation arose under the old mechanism; changing allocation or prices can change future behavior and budget depletion.
5. **How do cancellations affect feasibility?** Reservations need expiry and idempotent release; otherwise inventory can be stranded or sold twice, while incentive rules must specify whether cancellation is strategic.

8. Comparison Tables

Feature	Online auction	Offline auction
Future information	Hidden/partial	Complete input known
Decisions	Sequential, often irrevocable	Joint optimization possible
Main analysis	Competitive/Bayesian online guarantee	Welfare/revenue and truthfulness
State	Inventory, time, budgets	Full-instance constraints
Competitive ratio	Price of Anarchy	
Loss from lack of future knowledge	Loss from selfish equilibrium	
Compares algorithm to offline optimum		Compares equilibrium to social optimum

9. Common Mistakes

- Defining “online” as web-based rather than sequential-information constrained.
- Omitting the arrival/adversary model.
- Comparing with an infeasible offline benchmark.
- Assuming per-round truthfulness implies dynamic truthfulness.
- Ignoring atomic inventory and budget updates in system design.

10. Edge Cases / Special Cases

- Unknown horizon changes secretary-style thresholds.
- Zero or tiny inventory makes exploration expensive.
- Ties need deterministic or report-independent randomized handling.
- Late events, retries, and duplicate bids require idempotency.
- Correlated arrivals can invalidate guarantees based on independent distributions.
- Strategic buyers may delay arrival when timing is controllable.

11. How to Explain in Interview

“An online auction must allocate and price items as opportunities arrive, without knowing the future. I first state the arrival model and offline benchmark, then analyze welfare or revenue, truthfulness, and feasibility. Posted prices are a common practical tool, while competitive ratios quantify the value lost by online uncertainty.”

12. Quick Revision Notes

- Online = sequential decisions with hidden future, not “on a website.”
- Always state adversarial, random-order, or stochastic arrivals.
- Benchmark must obey the same inventory and capacity constraints.
- Posted price independent of current report gives simple truthfulness.
- Trap: competitive ratio and PoA measure different sources of loss.

13. Practice Tasks

1. Simulate the $1/e$ secretary algorithm and estimate success probability.
2. Implement an idempotent inventory reservation state machine.
3. Compare fixed and dynamic posted prices on synthetic arrivals.
4. Define a valid offline benchmark for budgeted ad allocation.
5. Explain why historical auction replay is not fully counterfactual.

14. Final Cheat Sheet

Item	Recall
Core definition	Sequential allocation/pricing without future knowledge
Why it matters	Real marketplaces operate under time, budget, and inventory limits
Most asked	Arrival models, competitive ratio, posted prices, secretary, pacing
Key comparison	Online uncertainty vs strategic inefficiency
One-line answer	“An online auction balances truthful immediate decisions against unknown future demand.”